# Document for Pokedex

2025-11-20

# Contents


1 The Pokedex Project ... 3
1.1 Introduction ... 3
1.2 What's Coming ... 4
1.3 Suggest Reading ... 4
1.4 Compatibility ... 4
1.5 System Requirement ... 4
2 ASL Model ... 5
2.1 Quick Glance at the Build Process ... 5
2.2 How we "decode" ... 5
2.2.1 Instructions ... 5
2.2.2 Control and Status Register ... 7
2.3 Stepping Instructions ... 7
2.4 Error handling ... 8
2.5 Architecture States ... 9
2.5.1 Program Counter ... 9
2.5.2 General Purpose Register ... 10
2.6 Extensions ... 11
2.6.1 `rv32_i` ... 11
2.6.1.1 `addi` ... 11
2.7 FFI and the Platform ... 11
3 Simulator ... 12
3.1 Execution ... 12
3.2 Device Bus ... 12
3.3 Commit Events ... 12
3.4 Configurations ... 12
4 Differential Test ... 13
4.1 Execution ... 13
4.2 Spike Hacks ... 13
4.3 Interfaces ... 13
5 Appendix A: Coding Style ... 14
6 Appendix B: ASLc Compiler Passes ... 15
7 Appendix C: How to build this document ... 16

# 1 The Pokedex Project

## 1.1 Introduction

Pokedex is an implementation of the RISC-V instruction set architecture (ISA). It provides a simple, maintainable, and flexible ISA design that allows developers to easily add new instructions or make architectural changes—capabilities that are often difficult to achieve with existing tools like `riscv/sail-riscv` or Spike (`riscv/riscv-isa-sim`).

The Pokedex project comprises three main components:

- A formal specification written in the ARM Architecture Specification Language (**The ISA Model**).
- A **simulator** written in Rust, which provides a system platform for co-simulation with the ISA model.
- A **differential test framework** to cross-validate architectural states against Spike and RTL.

You can think of *The ISA Model* as the code that implements the logic defined by the RISC-V ISA Specification. *The Simulator* acts as the host environment; it handles the necessary system-level operations (such as loading ELF files) that are not covered by the ISA specification but are required to run programs. Finally, the *Differential Test Framework* provides the tools necessary to validate and guarantee the correctness of *The ISA Model*.

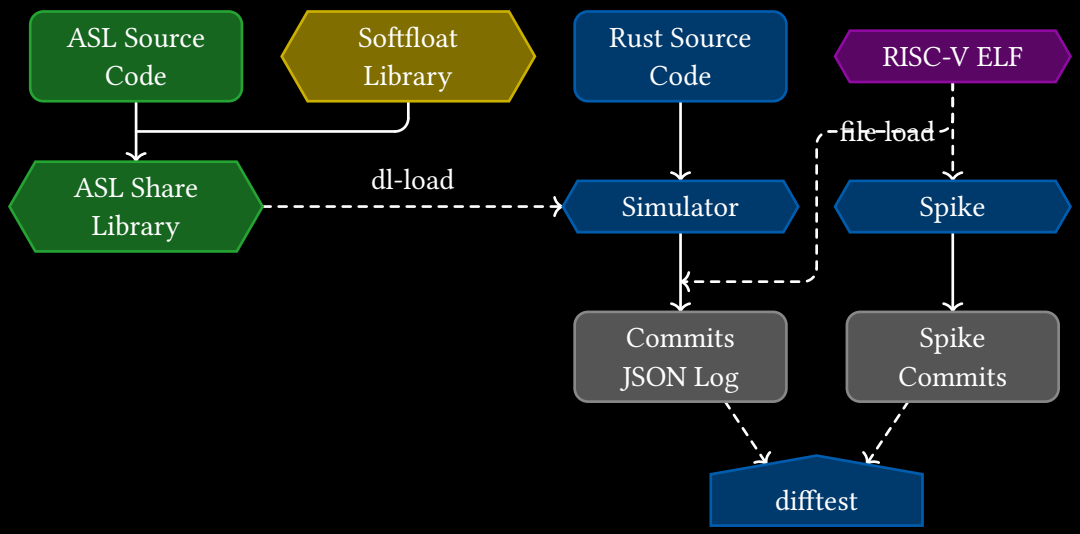


Figure 1: Pokedex Architecture

The primary goal of the Pokedex project is to serve as a "source of truth" for the ISA's behavior. This enables co-simulation to identify bugs in Register Transfer Level (RTL) designs.

> **Why the name "Pokedex"?**
>
> Our T1 testbench supports a collection of micro-architecture designs, each named after a Pokémon. We chose the name "Pokedex" to align with this theme.

> In the Pokémon world, a Pokédex is an essential tool for understanding the creatures you interact with. In that same spirit, this project provides the tools necessary to understand, maintain, and improve our T1 architectures.

## 1.2 What's Coming

This document is intended for readers with a basic understanding of software programming. It covers the entire Pokedex architecture and explains key design choices. By the end of this guide, readers will be able to modify and verify the whole project.

## 1.3 Suggest Reading

- **ASL Document** hosted by Intel Lab: https://intellabs.github.io/asl-interpreter/asl_reading.html
- **ASL Specification**: https://developer.arm.com/documentation/ddi0626/latest
- **RISC-V ISA Specification**: https://github.com/riscv/riscv-isa-manual/
- **ASL Prelude Reference**: https://github.com/IntelLabs/asl-interpreter/blob/master/prelude.asl
- **herdtools7** (The "official" ASL): https://github.com/herd/herdtools7/tree/ASLRefALP3.1/asllib

## 1.4 Compatibility

Note that we currently use the Intel Labs fork of the ASL Interpreter, which deviates slightly from the official ASL specification. A migration to *herdtools7* is planned, pending support for C code generation in that toolchain.

## 1.5 System Requirement

We use Nix as our primary build system. It orchestrates the build systems for each module and produces final artifacts without requiring manual dependency management.

The only requirement is a system with Nix installed. You can follow the installation guide here: https://nixos.org/download. (Note: You do not need the full NixOS operating system; only the Nix package manager CLI and daemon are required).

After installation, add the following configuration to `~/.config/nix/nix.conf`:

```
extra-experimental-features = flakes nix-command pipe-operators
```

Writing Nix code is outside the scope of this guide. However, the Pokedex project configuration is stable, so you generally will not need to modify Nix files. If you encounter build issues, please file an issue report.

# 2 ASL Model

We use ARM's Architecture Specification Language (ASL) to formally describe the RISC-V ISA. The core logic is organized within the `model/` directory:

- `aslbuild/`: Contains configuration files for `asl2c` to generate C code.
- `csr/`: Contains code snippets for individual Control and Status Register (CSR) implementations.
- `csrc/`: Contains C source code for ABI compatibility and module integration.
- `data_files/`: Pre-generated files containing deserializable data for the decoder.
- `extensions/`: Holds code snippets defining the semantics for each instruction, organized by ISA extension.
- `handwritten/`: Includes foundational, manually written code, such as architectural state declarations and helper libraries.
- `scripts/`: Includes scripts to generate Ninja build files, data files, etc.
- `template/`: Legacy code and templates, will be eliminate after migration.

## 2.1 Quick Glance at the Build Process

ASL provides limited support for polymorphic types. Consequently, implementing the full RISC-V ISA manually is difficult due to the significant amount of redundant logic required for instruction execution. To address this, we use Jinja to define and reuse code snippets.

The script `scripts/buildgen.py` acts as the primary build entry point. It scans all source files and generates a Ninja build file to perform the following tasks:

1. Process all `.j2` templates using JSON data from `data_files` to generate the final ASL code.
2. Combine the generated ASL code with the handwritten ASL code, then use the `asl2c` tool to compile them into C source code.
3. Compile the resulting C code alongside the interface code in `csrc/`, linking them with the `softfloat` library to produce the final dynamic library (`libpokedex_model.so`).

Now, we will step through the build process to explain the ASL model structure.

## 2.2 How we "decode"

Fundamentally, an instruction can be viewed as a sequence of bytes representing a "command" (opcode) paired with "arguments" (operands). The **decoder's** responsibility is to interpret this sequence: it identifies which "command" to execute and extracts the necessary "arguments", ensuring strict adherence to the ISA specification.

### 2.2.1 Instructions

Instruction encodings are defined in the `data_files/inst_encoding.json` file, which is generated by `scripts/datagen.py`. Each instruction entry requires the following fields to define an encoding:

```
{
  "name": "addi",
  "encoding": "-----------------000-----0010011",
  "extension": "rv_i"
}
```

The encoding data is derived from the riscv/riscv-opcodes project. We vendor the upstream parsing tool at `scripts/riscv_opcodes_util.py`, allowing us to accept any *riscv-opcodes* source. This enables us to easily define encodings for unratified extensions.

The encoding field is utilized by the `templates/inst_dispatch.asl.j2` template, which defines the instruction decoding entry point. Each encoding string becomes a bit-pattern match arm. When an instruction matches a specific bit pattern, the `DecodeAndExecute` function delegates execution to the corresponding function identified by the name field. If no bit pattern matches, an `IllegalInstruction` result is returned.

```
// example of code generated inst_dispatch.asl
func DecodeAndExecute(instruction : bits(32)) => Result
begin
  case instruction of
    when '-----------------000-----0010011' =>
      return Execute_ADDI(instruction);

    // ... other bit patterns and dispatcher ...

    otherwise =>
      return IllegalInstruction();
  end
end
```

In the example above, the decoder is defined as a function. A typical function block follows this structure:

```
function FunctionName(argument : type)
begin
  // function body
end
```

Pattern matching is achieved using the `case...when` expression. Each `when` keyword defines a match arm; if the pattern matches the value, the associated statements are executed. The `otherwise` keyword acts as a "catch-all" default arm; if no pattern matches the value, the statements defined in `otherwise` will be executed.

```
// ...
  case <expression> of
    when <pattern> =>
      <statement>
    otherwise =>
      <statement>
  end
// ...
```

As the number of supported ISA extensions grows, hand-writing these pattern matches becomes unmanageable. That's why we use Jinja to parse the list of encodings and automatically generate this instruction dispatch logic.

Details of `Result` and `IllegalInstruction` can be read at Section 2.4.

### 2.2.2 Control and Status Register

CSR handling follows the same logic as instruction decoding: a CSR definition file feeds an ASL Jinja template to generate the pattern matching and dispatch code.

The data is derived from the `riscv-opcodes` project and stored in `data_files/csr.json`. Each CSR is defined using the following JSON format:

```
{
  "name": "fflags",
  "mode": "urw",
  "addr": 1,
  "bin_addr": "000000000001",
  "read_write": true
}
```

This JSON data is processed by the `templates/csr_dispatch.asl.j2` template to generate CSR operations as follows:

```
func ReadCSR(csr : bits(12)) => CsrReadResult
begin
  case csr of
    when '000000001000' =>
      return Read_VSTART();

    // other csr

    otherwise =>
      return CsrReadIllegalInstruction();
  end
end

func WriteCSR(csr : bits(12)) => Result
begin
  case csr of
    when '000000001000' =>
      return Write_VSTART();

    // other csr

    otherwise =>
      return IllegalInstruction();
  end
end
```

Note that the `Write_XXX` function call is only generated if the CSR is defined as writable (based on the `read_write` fields).

Details of `CsrReadResult` and `CsrReadIllegalInstruction` can be read at Section 2.4.

## 2.3 Stepping Instructions

We have seen how instructions are decoded and executed. Next, we examine how instructions are fetched and passed to the decoder.

The `Step` function, defined in `handwritten/step.asl`, serves as the primary entry point for driving the model. It processes instructions sequentially, one at a time. Below is a simplified pseudo-code representation of this function:

```
// pseudo example code
func Step()
begin
  if HasInterrupt() then
    return;
  end

  let instruction = FFI_instruction_fetch(PC);
  FFI_issue_instruction(instruction);
  let result = DecodeAndExecute(instruction);
  if result.is_ok then
    FFI_commit_instruction(instruction);
  else
    HandleException();
  end
end
```

We will discuss how the Foreign Function Interface (FFI) interoperates with the model in Section 2.7. For now, treat any function with the `FFI_` prefix as a black box that performs the necessary external operations.

In each execution of Step, the model fetches a single instruction using the external FFI function. The `PC` (Program Counter) is a global architectural state variable (32-bit) representing the physical memory address of the current instruction. This `PC` value is passed to `FFI_instruction_fetch` to retrieve the instruction data at that specific address.

The execution flow is as follows:

1. `FFI_issue_instruction` is called to signal that an instruction is being issued.
2. The instruction data is passed to `DecodeAndExecute` (implemented in Section 2.2.1) to obtain the execution result.
3. If the result indicates success (no exceptions), `FFI_commit_instruction` is called to signal the instruction is finalized.
4. If an exception is raised, it is delegated to a separate exception handling function.

Note that the actual `Step` implementation handles significantly more complexity than shown above, including Compressed Instructions (the RISC-V "C" extension) and detailed exception logic. Please refer to `handwritten/step.asl` for the complete source code.

## 2.4 Error handling

ASL has poor polymorphic type support, and we don't want to use the `try...catch` error handling mechanism, so we defined a `Result` type to hold information and error for an operation:

```
record Result {
  is_ok : boolean;
  cause : integer{0..31};
  payload : bits(XLEN);
};
```

```
type CsrReadResult of record {
  data: bits(XLEN),
  result: Result
};
```

There are following API for creating a `Result`:

- `Exception(cause : integer{0..31}, payload : bits(N))`: return a `Result` with `is_ok` set to `FALSE` and `cause`, `payload` set to corrsponding arguments.
- `Retired()`: return a `Result` with `is_ok` set to `TRUE`, other fields undefined.
- `IllegalInstruction()`: return a `Result` with `is_ok` set to `FALSE`, `cause` set to illegal instruction error code, `payload` undefined.
- `ExceptionEcall(mode : PrivMode)`: return a `Result` with `is_ok` set to `FALSE`, `cause` set to machine ecall error code when argument `mode` is M mode, and `payload` undefined.
- `ExceptionEbreak(mode : PrivMode)`: return a `Result` with `is_ok` set to `FALSE`, `cause` set to machine breakpoint error code, and `payload` undefined.
- `CsrReadIllegalInstruction()`: return a `CsrReadResult` with `result` set to `IllegalInstruction()`, `data` undefined.
- `CsrReadOk(data : bits(XLEN))`: return `CsrReadResult` with `data` set to argument, `result` set to `OK`.

## 2.5 Architecture States

We have established the main logic for driving the model to process instructions sequentially. However, instructions rarely execute in isolation; they often exhibit **data dependencies**, where the current instruction requires the result of a previous one.

To manage this, the model requires persistent internal storage to hold these results between steps. This collection of information is known as the **Architectural State**.

These states are defined in file `handwritten/states.asl` and `handwritten/states_v.asl`.

### 2.5.1 Program Counter

The **Program Counter (PC)** is a fundamental architectural state that stores the address of the current instruction. It is defined in `handwritten/states.asl` as a 32-bit variable:

```
var __PC : bits(32);

getter PC => bits(32) begin return __PC; end

setter PC = npc : bits(32)
begin
  assert npc[0] == '0';
  __PC = npc;
end
```

To access the program counter, developers should utilize the public `PC` interface rather than the internal `__PC` variable. The `PC` setter enforces validity checks, ensuring that every address stored remains properly aligned (specifically, asserting that the least significant bit is zero).

**Example:**

```
// Read is always OK
let result = FFI_fetch_instruction(PC);

// Write OK
PC = 0x8000_0000[31:0];
// Write Panic
PC = 0x8000_0003[31:0];
```

### 2.5.2 General Purpose Register

The ASL Model currently supports the **RV32I** ISA. This defines 32 General Purpose Registers (GPRs), each 32 bits wide (`XLEN=32`). Since register `x0` is hardwired to `0` by definition, we optimize storage by allocating only 31 registers (`x1` through `x31`) within the model.

```
var __GPR : array[31] of bits(32);

getter X[i : XREG_TYPE] => bits(32)
begin
  if i == 0 then
    return Zeros(32);
  else
    return __GPR[i - 1];
  end
end

setter X[i : XREG_TYPE] = value : bits(32)
begin
  if i > 0 then
    __GPR[i - 1] = value;

    // notify emulator that a write to GPR occur
    FFI_write_GPR_hook(i, value);
  end
end
```

The `__GPR` variable is an array of 31 32-bits elements. Developers should exclusively use the public `X[i]` interface rather than accessing the `__GPR` variable directly.

The `X[i]` interface performs three critical tasks:

- It handles the index offset (mapping logical register `x1` to array index `0`).
- It enforces the hardwired zero behavior for `x0`.
- It signals the FFI hook whenever a write operation occurs.

**Example:**

```
assert(X[0] == Zeros(32));

X[5] = 0x8000_0000[31:0];
assert(X[5] == '10000000000000000000000000000000');
```

## 2.6 Extensions

### 2.6.1 `rv32_i`

#### 2.6.1.1 `addi`

## 2.7 FFI and the Platform

TODO

# 3 Simulator

## 3.1 Execution

## 3.2 Device Bus

## 3.3 Commit Events

## 3.4 Configurations

# 4 Differential Test

## 4.1 Execution

## 4.2 Spike Hacks

## 4.3 Interfaces

# 5 Appendix A: Coding Style

# 6 Appendix B: ASLc Compiler Passes

# 7 Appendix C: How to build this document